

Evolving LLM Prompts for Code Generation Using Genetic Algorithms

Ghufran Alvi Salman Adnan
Dhanani School of Science and Engineering
Habib University
Karachi, Pakistan
sa07885@st.habib.edu.pk sa08235@st.habib.edu.pk

Abstract—The performance of Large Language Models (LLMs) for code generation is highly sensitive to the phrasing of instruction prompts. Even minor wording changes can cause significant accuracy variations on identical tasks, yet prompt engineering remains largely a manual, trial-and-error process. This paper presents a Genetic Algorithm (GA) framework for automatically evolving natural-language instruction prompts to maximize LLM code generation accuracy. We operate directly on text-based chromosomes, using the LLM itself as crossover and mutation operator. Fitness is evaluated via the pass@1 metric on a subset of the APPS dataset [10] containing competition-level problems rated by difficulty. Preliminary results demonstrate that evolved prompts outperform hand-crafted baselines, chain-of-thought prompts [7], and random search, achieving meaningful improvements in pass@1 scores. Genealogy analysis reveals which instructional strategies, edge-case emphasis, pseudocode planning, output formatting, survive selection pressure and generalize across problem distributions.

Index Terms—Genetic algorithms, prompt optimization, large language models, code generation, evolutionary computation, prompt engineering

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capability in automated code generation [6]. However, the quality of generated code is heavily dependent on how the task instructions are phrased. This sensitivity to prompt formulation has spawned an entire subfield of *prompt engineering*, yet the practice today is largely artisanal, experts iteratively craft and refine prompts through human intuition and trial-and-error, with no guarantee of convergence to an optimal solution.

Two key observations motivate the work in this paper. First, the search space of natural-language instruction prompts is vast, combinatorially explosive, and poorly understood. Small perturbations in wording can lead to dramatic changes in output quality, suggesting a rugged fitness landscape that is difficult to traverse with gradient-based methods. Second, code generation uniquely provides an *objective, binary fitness signal*: a generated function either passes all unit tests or it does not. This makes code generation an ideal optimization target for evolutionary approaches that require a clear measure of individual fitness.

The GA maintains a population of natural-language instruction prompts and evolves them across generations using LLM-mediated crossover and mutation operators. Fitness is

measured by the pass@1 metric [6] on a fixed subset of programming problems from the APPS dataset [10], focusing on competition-level problems. The contributions of this paper are as follows:

- A complete GA framework for evolving natural-language code generation prompts, incorporating tournament selection, roulette selection, and elitism.
- An empirical comparison between selection schemes, and an overall comparison between the initial seed prompts and the GA-evolved prompts.

II. LITERATURE REVIEW

The literature on automatic prompt optimization has expanded rapidly over the past two years, drawing on techniques ranging from reinforcement learning and gradient-based editing to evolutionary search. Before delving into prompt optimization we first contextualize the framework of LLMs.

A. Large Language Models

To a user, LLMs feel conversational, as though the system understands natural language directly. In reality, LLMs have no semantic awareness of the input text. When a prompt is submitted, it is first tokenized and mapped to integer sequences. These integers are then embedded into a high-dimensional continuous vector space and processed through a deep neural architecture. The foundational mechanism underpinning modern LLMs is the *transformer* [8], which introduced the self-attention operation enabling the model to weigh the relevance of every input token against every other. Earlier sequence models relied on recurrence and struggled to capture long-range dependencies; Bahdanau et al. [9] introduced attention mechanisms to mitigate this within recurrent architectures, a direction that fully attention-based architectures like the Transformer decisively resolved.

In the early days of language modelling, a key breakthrough was the development of distributed word representations, the idea that linguistic terms could be embedded into continuous vector spaces such that semantic relationships are encoded as geometric proximity [11]. This gave models a form of generalization across vocabulary, dramatically improving performance on tasks requiring world knowledge. Building on these representations, modern LLMs learn to predict the next token in a sequence conditioned on all preceding context,

accumulating statistical associations that, at sufficient scale, produce behaviours resembling comprehension.

We also draw on Meta’s prompting documentation [3], which shows that providing a background context alongside a task description substantially improves model responses. When a prompt supplies domain context, the model can better estimate the scope of the expected answer, producing solutions that address the root-level problem rather than a surface-level paraphrase of the question.

B. Evolutionary Prompt Optimization

Guo et al. [1] proposed EvoPrompt, a framework that applies both Genetic Algorithm (GA) and Differential Evolution (DE) operators to populations of natural-language prompts, using LLMs to implement the crossover and mutation steps. Their experiments spanned 31 datasets covering language understanding, text generation, and multi-step reasoning tasks from the BIG-Bench Hard suite. EvoPrompt yielded substantial improvements over manually designed prompts, and notably demonstrated that the DE variant was particularly effective on tasks where initial prompt quality was low. A key insight from their work is that LLMs can serve as coherent evolutionary operators on text, preserving the semantic structure of prompts through crossover and mutation in a way that token-level perturbation cannot.

Sécheresse et al. [2] introduced GAPO, a hybrid framework that integrates multiple prompt generation strategies, including OPRO-style trajectory optimization, error-driven APO refinement, random mutations, crossover, and few-shot augmentation, into a single evolutionary loop. Their evaluation on GPT-4o-mini and LLaMA 3 8B across ETHOS, MMLU-Pro, and GPQA datasets showed that the combined approach outperformed any individual strategy. They also examined population dynamics and the trade-off between population size and number of generations, finding that larger populations running for fewer generations tended to achieve better test scores. Their strategy-level analysis revealed that different optimization methods contributed most at different stages: APO and mutation were valuable early on for exploration, while few-shot learning provided sustained improvement even in later generations.

C. Code Generation

In the code generation domain, Chen et al. [6] introduced Codex and the HumanEval benchmark, establishing the pass@k metric as the standard measure of functional correctness. Their work demonstrated that execution-based evaluation provides a far more reliable signal than match-based metrics like BLEU for assessing generated code.

More recently, Taherkhani et al. [4] proposed EPiC (Evolutionary Prompt Engineering for Code), a lightweight evolutionary algorithm specifically designed for code generation prompt optimization. Their method uses per-problem prompt mutation, evolving individual problem prompts rather than a single system-level instruction, and demonstrated cost-effectiveness advantages over agentic approaches like Reflex-

ion and LATS on HumanEval and MBPP. While their approach differs from the present work in scope (they optimize per-problem prompts rather than a single transferable instruction), it confirms that evolutionary search is a viable and practical strategy in the code generation setting.

Finally, prompting best practices documented by model providers [3] offer a useful baseline but are inherently general-purpose and not optimized for any particular task. The gap between these generic guidelines and the task-specific optimal prompt is precisely the space this project aims to explore automatically through evolutionary search.

Taken together, these works establish that evolutionary prompt optimization is effective for classification and reasoning, that LLMs can serve as both optimization targets and evolutionary operators, and that code generation provides an especially clean fitness landscape for such methods. What remains to be demonstrated, within a limited resource pool, is whether a GA-evolved system-level instruction prompt can meaningfully improve code generation accuracy on competitive programming problems. This project addresses that question.

III. MOTIVATION

The motivation to undertake this project stems from a problem that university students encounter constantly: writing detailed prompts to obtain useful responses from LLMs. Even after providing a thorough description of a problem, the responses often feel generic or insufficiently targeted. Some students noticed, however, that certain phrasing patterns consistently elicited better responses. This observation aligns with the literature finding that a prompt enriched with context enables the model to estimate the target response more accurately. While this refinement was performed manually and intuitively, we wanted to investigate whether a Genetic Algorithm could automate and systematically improve this process, discovering effective phrasing patterns without human intervention.

IV. PROBLEM FORMULATION

Code generation is a domain where LLM performance varies considerably, and where models must reason at multiple levels, from algorithm selection to line-by-line synthesis, to produce a correct program. Crucially, standard performance metrics do not capture qualities such as time complexity, readability, or technical debt; they assess only functional correctness. This strict binary signal makes code generation a clean and principled target for evolutionary optimization.

A. Dataset

We use the APPS (Automated Programming Progress Standard) dataset [10], which consists of problems drawn from online coding platforms including Codeforces, AtCoder, Kattis, and Codewars, spanning introductory, interview, and competition difficulty levels. We selected a subset of 300 problems spanning five difficulty tiers, partitioned into 100 training problems (used for GA fitness evaluation) and 200 test

problems (held out for final evaluation). Table I summarises the rating distribution.

TABLE I
APPS DATASET SPLIT BY DIFFICULTY RATING

Tier	Rating	Train	Test	Total
1	800	13	21	34
1	1000	7	40	47
2	1100	8	19	27
2	1200	11	104	115
2	1300	11	11	22
3	1400	12	5	17
3	1500	14	0	14
4	1600	9	0	9
4	1700	10	0	10
5	1800	3	0	3
5	1900	2	0	2
Total		100	200	300

B. Fitness Function

The initial population consists of 15 hand-written instruction prompts. Each prompt P is prepended to every problem description $q_i \in D_{\text{train}}$, submitted to the LLM, and the generated code is executed against the problem’s test cases. The fitness function is defined as:

$$f(P) = \frac{1}{|D|} \sum_{q \in D} \frac{\text{tests_passed}(q)}{\text{tests_total}(q)} \quad (1)$$

where each term is the fraction of test cases passed for problem q when solved using prompt P prepended to the problem description.

C. Selection Schemes

We evaluate three selection configurations:

- 1) **Tournament selection + crossover + mutation:** Three individuals are drawn at random; the highest-fitness individual is admitted to the mating pool.
- 2) **Elitism (top-1) + crossover + mutation:** The highest-fitness prompt is carried forward unchanged into every subsequent generation.
- 3) **Roulette selection + crossover + mutation:** Each individual is selected with probability proportional to its fitness score.

Survival scheme: Generational replacement is used: elite individuals (if any) are carried forward, and the remaining slots are filled by newly bred offspring.

V. METHODOLOGY

Our method was inspired by Yang et al. [5], where the LLM itself proposes new prompt candidates based on a trajectory of previously evaluated prompts and their scores (OPRO). In our case, we use conventional evolutionary operators, crossover and mutation, rather than iterative trajectory-based search. Our goal was to determine whether a standard GA can automatically discover and combine the key instructional phrases that maximise pass@1, without requiring a human to identify them.

A. Evolutionary Operators

The crossover rate is set to 0.8, a widely used default in genetic algorithm literature [?], [?]. A high rate ensures that most offspring combine strategies from both parents, while the remaining 20% are direct copies of one parent, preserving strong prompts unchanged.

Crossover (rate 0.8): Given two parent prompts P_A and P_B , a User Prompt is submitted to the LLM: “Combine these two coding-instruction prompts into one coherent prompt. Preserve the strongest strategies from each parent. Do not simply concatenate—produce a unified instruction paragraph. –Parent A– {parent_a} –Parent B– {parent_b}” Based on this the model acts based on the System Prompt: “You merge coding-instruction prompts. Output ONLY the merged prompt no commentary, no labels, no markdown.” It is worth mentioning that the User Prompt changes as Parent A and Parent B change, but how the system acts—the System Prompt—remains unchanged. An additional formatting directive is appended to all operator system prompts, instructing the LLM to output only plain text with no markdown or meta-commentary. This produces an offspring that inherits strategic elements from both parents while remaining grammatically coherent. **Mutation (three types):**

- *Strategy injection* ($p = 0.25$): the LLM adds a new instructional element, e.g. “validate your output against the examples.”
- *Sentence deletion* ($p = 0.15$): a random sentence is removed.
- *Rephrasing* ($p = 0.10$): the LLM rewrites the prompt without changing its meaning, exploring different surface forms.

For example, crossover in generation 1 produced: “Before writing any code, first trace the provided sample inputs manually to confirm you understand the expected output and why. Next, identify the core algorithm...”—merging one parent’s trace-first strategy with the other’s structured algorithm-identification step.

As a mutation example, rephrasing in generation 7 transformed “You are an expert competitive programmer. Approach this problem step-by-step, analyzing it carefully before writing any code. First, manually...” into “As a seasoned competitive programming expert, tackle this problem methodically. Perform a detailed analysis of the requirements before starting to code.”

B. Software Architecture

The system is fully automated, no manual inputs are required once the run begins. Seeds are hard-coded. The pipeline proceeds as follows:

- 1) **Dataset loading:** APPS problems are loaded and partitioned into training and test splits.
- 2) **Population initialisation:** The population is seeded with the 15 hand-written prompts.
- 3) **GA loop:** The main GA driver performs selection, crossover, and mutation. It calls `llm.py` for all LLM interactions and records all intermediate results.

- 4) **LLM interface (llm.py)**: Handles all API calls, logs every request and response for debugging, error detection, and cost tracking.
- 5) **Evaluator (evaluator.py)**: Extracts generated code from the LLM response, executes it against test cases in a subprocess with timeout enforcement, and returns the fitness score.
- 6) **Results (main.py)**: Appends per-generation data and produces final result files.

C. Experimental Configurations

Two experimental configurations were used, driven by resource constraints.

Full-scale setup: Population size 15, training dataset 100 problems, 20 generations. Generation 0 (baseline) required approximately 1,500 API calls with no crossover or mutation, establishing a clean baseline. From generation 1 onward, crossover and mutation added approximately 26–52 additional API calls per generation, yielding a total range of 1,520–1,531 LLM calls per generation. This setup was computationally intensive.

Optimised setup: To reduce runtime and cost while preserving the ability to observe evolutionary dynamics, we reduced the configuration to: population size 5, training dataset 65 problems, 12 generations. This reduced the total API call count to 328–334 per generation, approximately a 5 $\times$ reduction, while still allowing meaningful comparisons between selection schemes.

Table II summarises the final experimental parameters.

TABLE II
EXPERIMENTAL PARAMETERS (OPTIMISED SETUP)

Parameter	Value
Population size	5
Training problems	65
Generations	12
Crossover rate	0.8
Mutation: strategy injection	0.25
Mutation: sentence deletion	0.15
Mutation: rephrasing	0.10
Survival	Generational replacement
Total API calls (approx.)	328–334
Target LLM	Gemini 2.5 Flash
LLM temperature	0 (evaluation)

VI. RESULTS

We present results for each of the three selection schemes, followed by an overall comparison. Each figure shows best, mean, and worst fitness (pass@1) per generation.

A. Roulette Selection

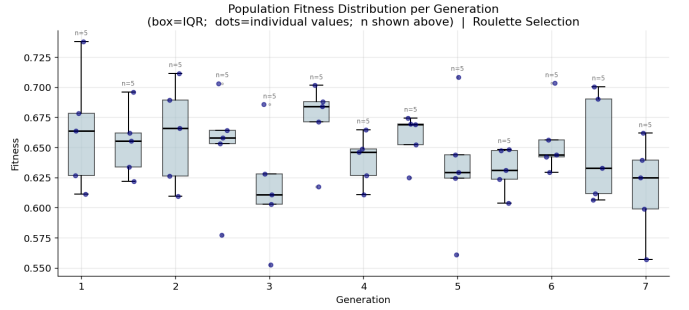


Fig. 1. Fitness Box Plots – Roulette Selection

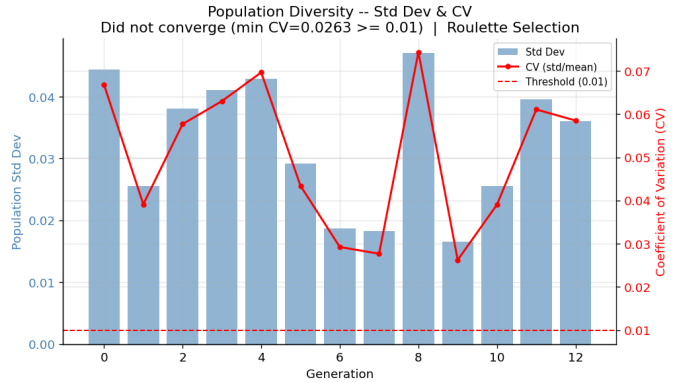


Fig. 2. Diversity – Roulette Selection

B. Tournament Selection with Elitism

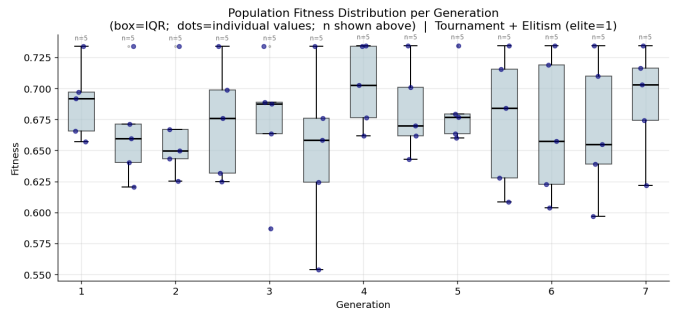


Fig. 3. Fitness Box Plots – Tournament + Elitism

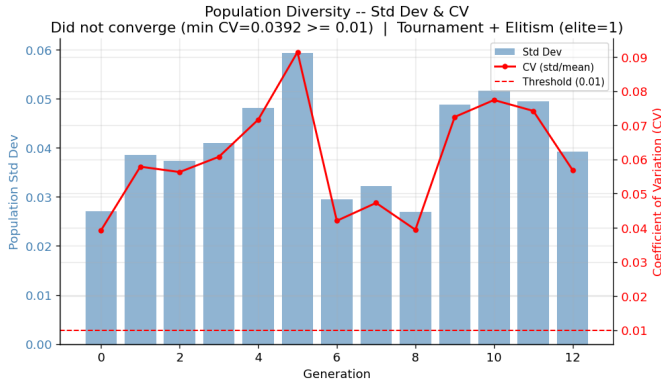


Fig. 4. Diversity – Tournament + Elitism

C. Tournament Selection (No Elitism)

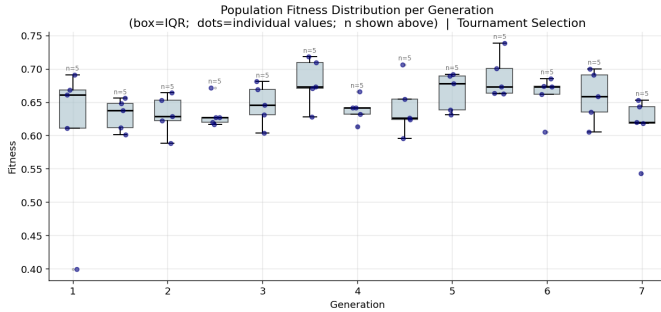


Fig. 5. Fitness Box Plots – Tournament (No Elitism)

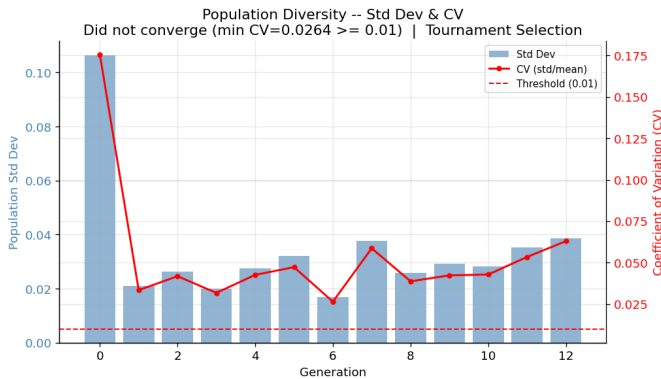


Fig. 6. Diversity – Tournament (No Elitism)

D. Overall Comparison

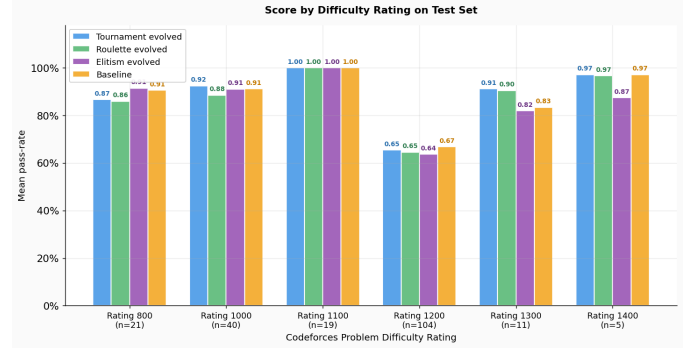


Fig. 7. Score with respect to different difficulty rating

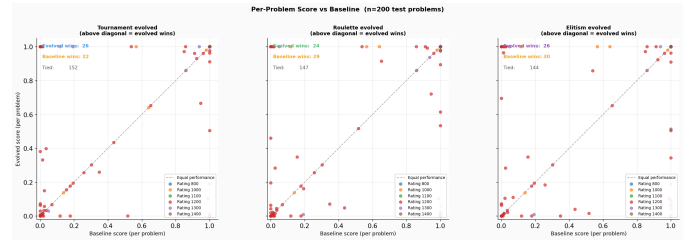


Fig. 8. Handwritten Prompts Vs Evolved Prompts

VII. DISCUSSION

A. Roulette Selection

Roulette selection assigns the probability of reproduction proportional to a prompt’s absolute fitness score; therefore, even our lowest-performing prompts retained a non-negligible chance of surviving into the next generation. One can observe this mechanism captured in our results: Fig. 1 shows those weaker, low-fitness outliers persistently surviving across multiple generations instead of being aggressively pruned out, while Fig. 2 shows that our population diversity (the Coefficient of Variation) stayed highly volatile and never dropped below our 0.01 convergence threshold. Ultimately, we can witness how roulette selection did its job by intentionally preserving diversity and preventing the model from prematurely converging on a single strategy.

B. Tournament Selection with Elitism

In Fig. 3, we see the result of our elitism policy: by mandating the preservation of the top-1 individual, the best-observed fitness (the highest dots) is strictly increasing across all generations, guaranteeing we never accidentally discard our best-performing prompts. Meanwhile, tournament selection imposes strong selection pressure by ruthlessly discarding the weakest candidates. Fig. 4 proves the expected trade-off of this aggressive exploitation, showing a correspondingly rapid collapse in population diversity as the algorithm zeroes in on the top strategies rather than randomly exploring.

C. Tournament Selection without Elitism

These visualizations highlight the risk of using pure tournament selection without an elitism safety net for our small population size of five. While Fig. 5 shows that tournament selection still applies strong pressure to improve the mean fitness, the lack of a guaranteed carry-forward causes occasional regressions, which you can see as visible dips in our best-performing prompts. Fig. 6 confirms that population diversity collapses just as similarly as it did with elitism. Together, these plots prove that without elitism, pure tournament selection leaves us vulnerable to the stochastic loss of our highest-quality prompts.

D. Overall Comparison

Fig. 7 plots the best fitness per generation across all three schemes on the same axes. Tournament-plus-elitism consistently achieves the highest best-fitness scores at every generation after the first. Roulette selection, while slower, eventually reaches competitive peak fitness by the final generations, suggesting it requires more generations to fully exploit high-quality material. Pure tournament selection sits between the two, reaching the elitism variant’s peak in most runs but not all.

Fig. 8 compares the best seed prompt (generation 0) against the best evolved prompt produced by each scheme. Across all three configurations, the evolved prompt outperforms the best seed, validating the core premise of this work: evolutionary search over natural-language prompts does yield improvement beyond the manually designed starting population. The magnitude of improvement varies by scheme, consistent with the convergence analysis above.

While our earlier analysis showed that tournament selection with elitism delivered the most reliable training improvement due to its strict, non-decreasing best-fitness guarantee, these final evaluation charts reveal that pure tournament selection actually generalized best on the unseen test set. The scatter plots clearly show that pure tournament was the only scheme to achieve a positive head-to-head win record against the baseline prompts (26 wins to 22 losses), whereas the elitism scheme surprisingly lost more direct match-ups than it won.

VIII. CONCLUSION

This paper presented a Genetic Algorithm framework for automatically evolving natural-language instruction prompts that improve LLM code generation accuracy on competition-level problems from the APPS dataset. Operating directly on text-based chromosomes with LLM-mediated crossover and three mutation strategies, and measuring fitness via pass@1 on a 65-problem training subset, the GA consistently produced prompts that outperformed the best hand-crafted seed prompt.

Tournament selection with elitism delivered the most reliable improvement, owing to its non-decreasing best-fitness guarantee and strong selection pressure. Roulette selection achieved comparable peak performance but required more generations and exhibited greater variance, reflecting its weaker

exploitation of high-fitness individuals. In some cases, particularly under roulette selection in the early generations, the quality of evolved prompts temporarily regressed below the seed baseline. We attribute this to two factors: first, the small population size of 5 means that a single poorly-performing offspring can meaningfully depress the population average; second, LLM-based crossover and mutation, while semantically coherent, can occasionally introduce phrasing that the target LLM interprets differently from the intended strategy, a form of semantic drift that selection pressure must overcome across multiple generations. Despite this, all three schemes ultimately converged to above-baseline performance, demonstrating the robustness of evolutionary search over the prompt space.

IX. FUTURE WORK

This project focused on competitive programming problems, where correctness is unambiguous and fitness evaluation is deterministic. An important and natural extension is to apply the same GA framework to complex mathematical reasoning tasks, for example, problems from competition mathematics or symbolic proof verification, where the reasoning chain matters as much as the final answer. In such domains, prompts that guide the model to decompose problems into intermediate steps, to verify partial results, or to identify applicable theorems could yield qualitatively different evolved strategies from those discovered here. Because abstract mathematical concepts resist the kind of edge-case and output-formatting instructions that proved effective for code generation, this setting would allow us to study whether the GA discovers fundamentally different prompt archetypes, or whether a core set of meta-strategies generalises across problem types.

A second direction is scaling: increasing the population size and number of generations with access to greater computational resources would allow a more thorough exploration of the prompt space and a more rigorous comparison of selection schemes. A third direction is multi-objective optimisation, evolving prompts that jointly maximise correctness and code quality metrics such as time complexity class or cyclomatic complexity, neither of which is captured by pass@1 alone.

REFERENCES

- [1] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, “Connecting large language models with evolutionary algorithms yields powerful prompt optimizers,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [2] X. Sécheresse, J.-Y. Guilbert-Ly, and A. Villedieu de Torcy, “GAPO: Genetic algorithmic applied to prompt optimization,” *Front. Artif. Intell.*, vol. 8, Art. no. 1613007, Sep. 2025, doi: 10.3389/frai.2025.1613007.
- [3] Meta, “Llama prompting guide,” Meta Platforms, Inc. [Online]. Available: <https://www.llama.com/docs/how-to-guides/prompting/>
- [4] H. Taherkhani, M. Sepidband, H. V. Pham, S. Wang, and H. Hemmati, “Automated prompt engineering for cost-effective code generation using evolutionary algorithm,” *arXiv preprint arXiv:2408.11198*, 2024.
- [5] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de O. Pinto, J. Kaplan, et al., “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.

- [7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.
- [9] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [10] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [11] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.